

Two Ways to Predict Survival

A Unified Theory of Linear and Tree-Based Models

Applied to the Titanic Dataset

Short Summary

We present a first-principles comparative study of two fundamental model families for binary classification—parametric linear models and non-parametric tree architectures—applied to the benchmark Kaggle Titanic passenger dataset ($n_{\text{train}} = 891, n_{\text{test}} = 418$). Six model configurations are developed from rigorous theoretical foundations: unregularized, Ridge (L_2), Lasso (L_1), and Elastic Net logistic regression, alongside a single decision tree and Random Forest. For logistic regression, we derive the cross-entropy loss from Bernoulli maximum likelihood and prove loss convexity (Theorem 2.3) to guarantee global convergence under gradient descent. Regularization variants are derived as explicit weight-space penalties (Remark 2.4). For tree models, node impurity and information gain are formally defined to prove that binary partitioning monotonically reduces expected impurity (Proposition 3.1), while Random Forest variance reduction is established via the analytical derivation of the 63.2% bootstrap inclusion rate (Lemma 3.1) combined with feature subsampling ($m_{\text{try}} = \lfloor \sqrt{p} \rfloor$). Empirical evaluation on unseen test passengers demonstrates high inter-model consensus (98.56% prediction agreement), with regularized linear models and the Random Forest ensemble both achieving top-tier test classification accuracy (0.77751), confirming that feature engineering effectively captured the core non-linear survival dynamics.

Contents

1	Introduction	2
1.1	The Motivating Problem	2
2	Logistic Regression as a Generalized Linear Model	2
2.1	The Linear Predictor and the Sigmoid	2
2.2	Maximum Likelihood and the Cross-Entropy Loss	4
2.3	The Gradient of the Cross-Entropy Loss	4
2.4	Convexity of the Cross-Entropy Loss	6
2.5	L2 and L1 Regularization	7
3	Tree-Based Ensemble Methods	10
3.1	A Single Decision Tree	10
3.2	Random Forest: Averaging Decorrelated Trees	12
4	Model Comparison: Decision Trees, Random Forests, and Logistic Regression	14
5	Discussion: Two Families, One Question	15
6	Empirical Validation on Real Data	15

1 Introduction

1.1 The Motivating Problem

Given a passenger’s ticket class, sex, age, family size, fare, and a set of engineered features, our goal is to predict binary survival in the historical sinking of the RMS Titanic. This benchmark binary classification task originates from the standard Kaggle Titanic competition, providing $n_{\text{train}} = 891$ labeled training observations alongside an unseen test evaluation set of $n_{\text{test}} = 418$ passengers. We evaluate six distinct model configurations spanning two fundamentally different algorithmic families:

Family	Models	Core Mechanism
Linear	Logistic Regression (No Penalty, L_1 , L_2 , Elastic Net)	Score mapping $z = w^\top x + b \rightarrow \sigma(z)$
Tree-based	Decision Tree, Random Forest	Axis-aligned recursive partitioning

The standard temptation when comparing parametric linear models and non-parametric trees is to treat them as isolated algorithms with disparate objectives, hyperparameter rules, and intuitions built from scratch. This work argues against that fragmented framing. Rather than forcing both into an artificial shared mold—Section 3 explicitly demonstrates that decision trees do *not* compute the linear score $z = w^\top x + b$ derived in Section 2—both families are subjected to the same rigorous theoretical standard:

1. What mathematical object is being optimized?
2. Why is that the principled object to optimize for binary classification?
3. What theoretical mechanism prevents the model from overfitting noise in the training distribution?

For logistic regression, this requires deriving the sigmoid, Bernoulli cross-entropy loss, and parameter gradients from first principles, proving loss convexity (Theorem 2.3) to guarantee global convergence under optimization, and establishing L_2 , L_1 , and Elastic Net regularization as formal bounds on model capacity. For tree-based models, it requires defining node impurity and Information Gain precisely enough to prove that binary splitting never increases expected impurity (Proposition 3.1), while establishing Random Forest variance reduction as an analytical consequence of bootstrap aggregation (Lemma 3.1) and randomized feature subsampling ($m_{\text{try}} = \lfloor \sqrt{p} \rfloor$).

Finally, both model families were evaluated on Kaggle’s test set of 418 unseen passengers. The Kaggle score simply measures classification accuracy—the percentage of passenger survival outcomes the model got right. This provides a direct test of whether linear regularization and tree ensembling successfully capture the real survival patterns on new data.

2 Logistic Regression as a Generalized Linear Model

The first approach we choose for predicting passenger survival on the Titanic is **logistic regression**. This section establishes the mathematical foundations of the model: from parameterization and maximum likelihood estimation to convex optimization and regularization.

2.1 The Linear Predictor and the Sigmoid

The training set consists of n passengers, $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, where:

- $y_i \in \{0, 1\}$ records whether passenger i survived ($y_i = 1$) or did not ($y_i = 0$);

- $x_i \in \mathbb{R}^p$ collects their features: ticket class, age, fare paid, number of siblings/spouses and parents/children aboard, and so on.

Stacking these gives $X = (x_1, \dots, x_n)^\top \in \mathbb{R}^{n \times p}$ and $y = (y_1, \dots, y_n)^\top \in \{0, 1\}^n$.

A weight vector $w \in \mathbb{R}^p$ and an intercept $b \in \mathbb{R}$ combine a passenger's features into the linear predictor:

$$z_i := w^\top x_i + b, \quad i = 1, \dots, n,$$

where w_j is how much feature j pushes z_i up or down — w_{Fare} , for instance, is the shift in z_i per one-unit increase in fare paid, holding the rest of x_i fixed — and b is a constant offset shared by every passenger.

To turn z_i into a probability of survival, we assume:

$$y_i \mid x_i \sim \text{Bernoulli}(p(x_i)), \quad p(x_i) := P(y_i = 1 \mid x_i) \in (0, 1), \quad \text{independently across } i.$$

Since $p(x_i) \in (0, 1)$ while z_i ranges over all of $\mathbb{R}$, the two are related through the *logit* of $p(x_i)$, its log-odds, which also ranges over all of $\mathbb{R}$:

$$\text{logit}(p(x_i)) := \log \frac{p(x_i)}{1 - p(x_i)} = z_i. \quad (1)$$

Proposition 2.1 (Sigmoid)

Equation (1) solved for $p(x_i)$ gives $p(x_i) = \sigma(z_i)$, where $\sigma(z) := 1/(1 + e^{-z})$.

Proof.

Taking the exponential of both sides of Equation (1) gives:

$$\frac{p}{1 - p} = e^z \implies p = e^z(1 - p) = e^z - pe^z.$$

Rearranging terms to isolate p :

$$p(1 + e^z) = e^z \implies p = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z). \quad \square$$

Proposition 2.1 establishes that $p(x_i) = \sigma(z_i)$: the true survival probability equals the sigmoid of the linear predictor. We name this quantity

$$\boxed{\hat{y}_i := \sigma(z_i) = \sigma(w^\top x_i + b)} \quad (2)$$

the **model's predicted survival probability**. By definition, $\hat{y}_i$ is computed from (w, b) and x_i alone. Substituting $\hat{y}_i$ for $p(x_i)$ in the assumption above yields the model used throughout this work:

$$y_i \mid x_i \sim \text{Bernoulli}(\hat{y}_i), \quad \text{independently across } i. \quad (3)$$

Lemma 2.1 (Sigmoid derivative)

A useful result for subsequent gradient (Theorem 2.2) and Hessian (Lemma 2.2) computations is:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Proof.

Differentiating $\sigma(z) = (1 + e^{-z})^{-1}$ via the quotient or chain rule yields:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}.$$

Noting that $1 - \sigma(z) = 1 - \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}}$, we obtain:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad \square$$

2.2 Maximum Likelihood and the Cross-Entropy Loss

Equation (2) defines $\hat{y}_i = \sigma(w^\top x_i + b)$ for an arbitrary choice of (w, b) ; it does not yet tell us which choice to make. We select (w, b) through the principle of maximum likelihood.

Definition 2.1 (Likelihood)

Given the model $y_i \mid x_i \sim \text{Bernoulli}(\hat{y}_i)$ with $\hat{y}_i = \sigma(w^\top x_i + b)$ independently across i , the *likelihood* of (w, b) is the probability assigned to the observed binary outcomes $y_1, \dots, y_n$ as a function of the parameters:

$$L(w, b) := \prod_{i=1}^n P(y_i \mid x_i) = \prod_{i=1}^n \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}.$$

The observed data (x_i, y_i) remain fixed throughout: only (w, b) vary, entering through $\hat{y}_i$. The *maximum-likelihood estimate* (**MLE**) is defined as:

$$(\hat{w}, \hat{b}) := \underset{w, b}{\operatorname{argmax}} L(w, b),$$

which identifies the parameters under which the observed survival outcomes were most probable.

Theorem 2.1 (Cross-entropy is the negative log-likelihood)

The MLE (Definition 2.1) is equivalent to finding parameters (w, b) that minimize the empirical *cross-entropy loss*:

$$\text{CE}(w, b) := -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right].$$

Proof.

By Definition 2.1, $L(w, b) = \prod_{i=1}^n \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$. Because the natural logarithm is a strictly monotonic increasing function, maximizing $L(w, b)$ yields the same optimal parameters as maximizing its log-likelihood:

$$\log L(w, b) = \sum_{i=1}^n \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right].$$

Maximizing $\log L(w, b)$ is strictly equivalent to minimizing its negative, $-\log L(w, b)$. Scaling by the positive constant factor $1/n$ leaves the location of the minimizer invariant while normalizing the objective into a dataset-size invariant sample average:

$$\underset{w, b}{\operatorname{argmax}} L(w, b) = \underset{w, b}{\operatorname{argmin}} \left(-\frac{1}{n} \log L(w, b) \right) = \underset{w, b}{\operatorname{argmin}} \text{CE}(w, b). \quad \square$$

2.3 The Gradient of the Cross-Entropy Loss

By Theorem 2.1, fitting the model corresponds to minimizing $\text{CE}(w, b)$. We achieve this via gradient descent: starting from an initial parameter configuration (w, b) , we iteratively step in the direction of steepest decrease, opposite to ∇CE .

To tidy up the notation, we combine w and b into a single parameter vector $\theta := (b, w^\top)^\top \in \mathbb{R}^{p+1}$ and augment the features matrix with a leading column of ones: $\tilde{X} := [\mathbf{1} \mid X] \in \mathbb{R}^{n \times (p+1)}$. Under this parameterization, the linear predictor for passenger i is:

$$z_i = \tilde{x}_i^\top \theta, \quad \text{where } \tilde{x}_i := (1, x_i^\top)^\top \in \mathbb{R}^{p+1}.$$

Theorem 2.2 (Gradient of the cross-entropy loss)

The gradient of the cross-entropy loss with respect to θ is:

$$\nabla_{\theta} \text{CE}(\theta) = \frac{1}{n} \tilde{X}^{\top} (\hat{y} - y).$$

Proof.

Express $\text{CE}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell_i$, where $\ell_i := -[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$ denotes the loss contribution of passenger i . Differentiating ℓ_i with respect to θ via the chain rule through $\theta \rightarrow z_i \rightarrow \hat{y}_i \rightarrow \ell_i$:

$$\frac{\partial \ell_i}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i} + \frac{1 - y_i}{1 - \hat{y}_i} = \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}.$$

By Lemma 2.1, $\frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i)$. Combining these yields:

$$\frac{\partial \ell_i}{\partial z_i} = \frac{\partial \ell_i}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} = \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} \cdot \hat{y}_i(1 - \hat{y}_i) = \hat{y}_i - y_i.$$

Since $\frac{\partial z_i}{\partial \theta} = \tilde{x}_i$, the gradient for individual i is $\frac{\partial \ell_i}{\partial \theta} = (\hat{y}_i - y_i) \tilde{x}_i$. Averaging across all n passengers:

$$\nabla_{\theta} \text{CE}(\theta) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \tilde{x}_i = \frac{1}{n} \tilde{X}^{\top} (\hat{y} - y). \quad \square$$

Corollary 2.1 (The gradient in (w, b) coordinates)

Writing $\theta = (b, w^{\top})^{\top}$ and $\tilde{X} = [\mathbf{1} \mid X]$, the gradient system in Theorem 2.2 splits into separate partial derivatives:

$$\frac{\partial \text{CE}}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i), \quad \frac{\partial \text{CE}}{\partial w} = \frac{1}{n} X^{\top} (\hat{y} - y).$$

Proof.

Writing $\tilde{X} = [\mathbf{1} \mid X]$ in block matrix form gives $\tilde{X}^{\top} = \begin{pmatrix} \mathbf{1}^{\top} \\ X^{\top} \end{pmatrix}$. Matrix-vector multiplication yields:

$$\tilde{X}^{\top} (\hat{y} - y) = \begin{pmatrix} \mathbf{1}^{\top} (\hat{y} - y) \\ X^{\top} (\hat{y} - y) \end{pmatrix}.$$

Since $\theta = (b, w^{\top})^{\top}$, scaling by $1/n$ separates the gradient into its partial derivative:

$$\begin{aligned} \frac{\partial \text{CE}}{\partial b} &= \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \\ \frac{\partial \text{CE}}{\partial w} &= \frac{1}{n} X^{\top} (\hat{y} - y) \end{aligned} \quad \square$$

Remark 2.1 (Absence of a closed-form solution)

Setting the gradient in Theorem 2.2 to zero yields the stationary condition:

$$\tilde{X}^{\top} (\hat{y} - y) = 0, \quad \text{where } \hat{y} = \sigma(\tilde{X}\theta).$$

In standard linear regression, the corresponding condition is $\tilde{X}^{\top} (\tilde{X}\theta - y) = 0$, which simplifies to the normal equations $\tilde{X}^{\top} \tilde{X}\theta = \tilde{X}^{\top} y$ and admits the analytical closed-form solution $\theta = (\tilde{X}^{\top} \tilde{X})^{-1} \tilde{X}^{\top} y$. For logistic regression, however, $\hat{y} = \sigma(\tilde{X}\theta)$ is non-linear, meaning parameter estimation requires iterative numerical optimization.

Definition 2.2 (Gradient descent)

Starting from an initial parameter vector $\theta^{(0)}$ (e.g., $\theta^{(0)} = \mathbf{0}$), gradient descent constructs a sequence of parameter updates:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha \nabla_{\theta} \text{CE}(\theta^{(t)}), \quad t = 0, 1, 2, \dots,$$

given a positive learning rate $\alpha > 0$, running until $\|\nabla_{\theta} \text{CE}\|$ falls below a chosen tolerance threshold or a maximum number of iterations is reached. Expressed in (w, b) coordinates via Corollary 2.1, the updates take the form:

$$w^{(t+1)} := w^{(t)} - \frac{\alpha}{n} X^{\top} (\hat{y}^{(t)} - y), \quad b^{(t+1)} := b^{(t)} - \frac{\alpha}{n} \sum_{i=1}^n (\hat{y}_i^{(t)} - y_i).$$

2.4 Convexity of the Cross-Entropy Loss

In practice, fitting the model requires running gradient descent until the gradient is numerically zero. However, a zero gradient ($\nabla_{\theta} \text{CE}(\theta) = 0$) indicates that θ is a critical point: it could represent a global minimum, a local minimum, a local maximum, or a saddle point. What rules out non-global extrema is convexity: for a convex function, every critical point is a global minimum. We verify this property by deriving the Hessian matrix and establishing that it is positive semi-definite everywhere.

Lemma 2.2 (Hessian)

The Hessian of $\text{CE}(\theta)$ is given by:

$$\nabla_{\theta}^2 \text{CE}(\theta) = \frac{1}{n} \tilde{X}^{\top} D \tilde{X}, \quad \text{where } D := \text{diag}(\hat{y}_1(1 - \hat{y}_1), \dots, \hat{y}_n(1 - \hat{y}_n)).$$

Proof.

The Hessian is the Jacobian matrix of the gradient $\nabla_{\theta} \text{CE}(\theta) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \tilde{x}_i$ (Theorem 2.2) with respect to θ . Because y_i and $\tilde{x}_i$ are fixed observations, only $\hat{y}_i = \sigma(\tilde{x}_i^{\top} \theta)$ depends on θ . Applying Lemma 2.1 alongside the chain rule yields:

$$\frac{\partial \hat{y}_i}{\partial \theta} = \hat{y}_i(1 - \hat{y}_i) \tilde{x}_i.$$

Differentiating the i -th term in the gradient sum with respect to $\theta^{\top}$ gives the outer product matrix:

$$\frac{\partial}{\partial \theta^{\top}} [(\hat{y}_i - y_i) \tilde{x}_i] = \tilde{x}_i \left(\frac{\partial \hat{y}_i}{\partial \theta} \right)^{\top} = \hat{y}_i(1 - \hat{y}_i) \tilde{x}_i \tilde{x}_i^{\top}.$$

Summing over all n samples and dividing by n :

$$\nabla_{\theta}^2 \text{CE}(\theta) = \frac{1}{n} \sum_{i=1}^n \hat{y}_i(1 - \hat{y}_i) \tilde{x}_i \tilde{x}_i^{\top}.$$

Because $\tilde{X}$ has i -th row $\tilde{x}_i^{\top}$ and D is diagonal with entry $D_{ii} = \hat{y}_i(1 - \hat{y}_i)$, expanding the matrix product $\frac{1}{n} \tilde{X}^{\top} D \tilde{X}$ term-by-term reproduces this outer product sum exactly. $\square$

Theorem 2.3 (Convexity)

$\text{CE}(\theta)$ is convex on $\mathbb{R}^{p+1}$.

Proof.

A twice-differentiable function is convex on $\mathbb{R}^{p+1}$ if and only if its Hessian matrix is positive semi-definite at every point θ , meaning $v^{\top} \nabla_{\theta}^2 \text{CE}(\theta) v \geq 0$ for all directions $v \in \mathbb{R}^{p+1}$.

Using Lemma 2.2, for any arbitrary non-zero vector $v \in \mathbb{R}^{p+1}$:

$$v^\top \left(\frac{1}{n} \tilde{X}^\top D \tilde{X} \right) v = \frac{1}{n} (\tilde{X}v)^\top D (\tilde{X}v) = \frac{1}{n} \sum_{i=1}^n \hat{y}_i (1 - \hat{y}_i) (\tilde{X}v)_i^2,$$

where $(\tilde{X}v)_i$ denotes the i -th scalar entry of the vector $\tilde{X}v$. Every element in this sum is non-negative: $(\tilde{X}v)_i^2 \geq 0$ by definition, and $\hat{y}_i(1 - \hat{y}_i) > 0$ for all predicted probabilities $\hat{y}_i \in (0, 1)$. Consequently, the sum satisfies:

$$v^\top \nabla_\theta^2 \text{CE}(\theta) v \geq 0, \quad \forall v \in \mathbb{R}^{p+1}, \forall \theta \in \mathbb{R}^{p+1}.$$

Thus, $\nabla_\theta^2 \text{CE}(\theta) \succeq 0$ ¹ everywhere, establishing that $\text{CE}(\theta)$ is convex. $\square$

2.5 L2 and L1 Regularization

Theorem 2.1 fits (w, b) by minimizing $\text{CE}(w, b)$ alone. Gradient descent under this unregularized loss optimizes weights to fit the training data as closely as possible: including noise specific to the sample rather than true population trends. Large weights are a common indicator of overfitting: while $w_{\text{Age}} = 0.3$ represents a plausible effect size, $w_{\text{Age}} = 8.7$ suggests the model relies excessively on age due to sample noise.

Regularization mitigates overfitting by adding a penalty term to the objective function, punishing large weight magnitudes. Two standard penalties are used in practice:

- **L2 (Ridge) penalty:** $\frac{\lambda}{2n} \|w\|_2^2$, which shrinks weights smoothly toward zero;
- **L1 (Lasso) penalty:** $\frac{\lambda}{n} \|w\|_1$, which enforces sparsity by driving weights to exactly zero.

For L2 regularization, the regularized cross-entropy loss function is defined as:

$$J_\lambda(\theta) := \text{CE}(\theta) + \underbrace{\frac{\lambda}{2n} \|w\|_2^2}_{\text{L2 penalty}}, \quad \lambda \geq 0,$$

Optimizing J_λ via gradient descent requires updating the gradient computations from Corollary 2.1.

Corollary 2.2 (Regularized gradient)

The partial derivatives of the L2-regularized objective $J_\lambda(\theta)$ with respect to w and b are:

$$\nabla_w J_\lambda = \underbrace{\frac{1}{n} X^\top (\hat{y} - y)}_{\text{cross-entropy term}} + \underbrace{\frac{\lambda}{n} w}_{\text{L2 penalty term}}, \quad \frac{\partial J_\lambda}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i).$$

Proof.

Because J_λ is additive, its gradient is the sum of the unregularized cross-entropy gradient (Corollary 2.1) and the derivative of the penalty term. Differentiating $\frac{\lambda}{2n} \|w\|_2^2 = \frac{\lambda}{2n} \sum_{j=1}^p w_j^2$ with respect to w_j yields $\frac{\lambda}{n} w_j$. Since the penalty contains no b term, $\partial J_\lambda / \partial b$ remains unchanged. $\square$

Remark 2.2 (Proportional penalty magnitude)

The L2 penalty term $\frac{\lambda}{n} w_j$ scales linearly with w_j : at $w_j = 5.0$, the shrinking force is 5 times greater than at $w_j = 1.0$, and 50 times greater than at $w_j = 0.1$. As $w_j \rightarrow 0$, the penalty term vanishes.

¹The notation $A \succeq 0$ denotes that the matrix A is positive semi-definite, meaning $v^\top A v \geq 0$ for all vectors $v \in \mathbb{R}^{p+1}$.

Proposition 2.2 (Weight decay)

Applying gradient descent (Definition 2.2) to J_λ with learning rate α yields the weight update step:

$$w^{(t+1)} := w^{(t)} \left(1 - \frac{\alpha\lambda}{n} \right) - \frac{\alpha}{n} X^\top (\hat{y}^{(t)} - y),$$

while the bias parameter b updates identically to the unregularized case.

Proof.

Substituting Corollary 2.2's gradient into $w^{(t+1)} := w^{(t)} - \alpha \nabla_w J_\lambda$:

$$w^{(t+1)} = w^{(t)} - \alpha \left(\frac{1}{n} X^\top (\hat{y}^{(t)} - y) + \frac{\lambda}{n} w^{(t)} \right) = w^{(t)} \left(1 - \frac{\alpha\lambda}{n} \right) - \frac{\alpha}{n} X^\top (\hat{y}^{(t)} - y). \quad \square$$

Each step updates w by scaling it by the decay factor $(1 - \frac{\alpha\lambda}{n}) \in (0, 1)$ before applying the cross-entropy gradient update. The bias b is left unregularized because w controls feature sensitivity, while b represents the baseline log-odds.²

Remark 2.3 (L1 / Lasso regularization)

Replacing the L2 penalty with the L1 norm yields the Lasso-regularized loss function:

$$J_\lambda(\theta) = \text{CE}(\theta) + \frac{\lambda}{n} \|w\|_1, \quad \text{where } \|w\|_1 = \sum_{j=1}^p |w_j|.$$

For non-zero weights ($w_j \neq 0$), the absolute value function is differentiable with $\frac{d|w_j|}{dw_j} = \text{sign}(w_j)$, yielding the partial derivative:

$$\frac{\partial J_\lambda}{\partial w_j} = \frac{1}{n} \left[X^\top (\hat{y} - y) \right]_j + \frac{\lambda}{n} \text{sign}(w_j).$$

At $w_j = 0$, $|w_j|$ is non-differentiable. Unlike L2 regularization (where the penalty derivative vanishes as $w_j \rightarrow 0$), the L1 penalty maintains a constant magnitude $\frac{\lambda}{n}$. Consequently, $w_j = 0$ remains an optimal stationary point whenever the empirical cross-entropy gradient does not exceed the regularization threshold:

$$\left| \frac{1}{n} \left[X^\top (\hat{y} - y) \right]_j \right| \leq \frac{\lambda}{n}.$$

When this condition holds, the regularization term outweighs the data gradient, holding w_j at zero and eliminating the feature from the model.

Remark 2.4 (Comparing L1 (Lasso) and L2 (Ridge) Regularization)

As noted in Remark 2.2, Ridge's gradient contribution $\frac{\lambda}{n} w_j$ weakens as $w_j \rightarrow 0$, vanishing at the origin: weights approach zero asymptotically but rarely reach it exactly. In contrast, Lasso's penalty contribution maintains a constant magnitude λ/n regardless of how small w_j becomes. This constant penalty rate drives non-informative weights directly to zero, performing implicit feature selection.

Remark 2.5 (Elastic Net regularization)

Combining both penalties produces the Elastic Net objective function:

$$J_{\lambda_1, \lambda_2}(\theta) = \text{CE}(\theta) + \frac{\lambda_1}{n} \|w\|_1 + \frac{\lambda_2}{2n} \|w\|_2^2.$$

²When all feature inputs are zero ($x = \mathbf{0}$), the model predicts $\hat{y} = \sigma(b)$, making $b = \log\left(\frac{\hat{y}}{1-\hat{y}}\right)$ the prior log-odds of the survival probability.

Property	L2 (Ridge)	L1 (Lasso)
Penalty term	$\frac{\lambda}{2n} \sum_{j=1}^p w_j^2$	$\frac{\lambda}{n} \sum_{j=1}^p w_j $
Gradient contribution	$\frac{\lambda}{n} w_j$	$\frac{\lambda}{n} \text{sign}(w_j) \ (w_j \neq 0)$
Penalty magnitude	Proportional to $ w_j $	Constant (λ/n)
Zeroing behavior	Asymptotic shrinkage toward zero	Exact zeroing (sparse weights)
Feature selection	No	Yes

This approach gives us the best of both methods. The L1 penalty selects important features by setting extra weights to zero, while the L2 penalty stabilizes the model when features are strongly related to each other. When two features are closely related, Elastic Net keeps both and shrinks their weights together rather than randomly picking just one. The only drawback is that two parameters must be tuned (λ_1 and λ_2) instead of one.

Remark 2.6 (Model selection criteria)

The choice between regularizers depends on the underlying structure of the features:

- **Ridge** is preferred when most features contribute to the target or when features are strongly correlated.
- **Lasso** is best suited for high-dimensional settings containing many irrelevant features, as it sets non-informative weights to zero to produce a sparse, interpretable model.
- **Elastic Net** serves as a balanced approach when working with high-dimensional data with strongly correlated features, setting irrelevant weights to zero while retaining groups of correlated predictors together.

3 Tree-Based Ensemble Methods

This section develops two non-parametric models based on recursive partitioning: a single decision tree and Random Forest (an ensemble of decorrelated decision trees, trained in parallel on bootstrap samples). Neither of these models uses the linear score $z = w^\top x + b$ from Section 2 directly; a tree instead partitions $\mathbb{R}^p$ into axis-aligned hyper-rectangles. While a single decision tree trades off bias and variance through structural stopping rules, Random Forests leverage ensembling to systematically drive down model variance without increasing bias.

3.1 A Single Decision Tree

Definition 3.1 (Node Impurity)

For a set of training points $\mathcal{S}$ at a given node,³ let

$$\hat{p} = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \mathbf{1}[y_i = 1]$$

denote the empirical survival rate within $\mathcal{S}$. We measure the heterogeneity (uncertainty) of $\mathcal{S}$ using one of two standard criteria:

- **Gini Impurity:** $G(\mathcal{S}) = 2\hat{p}(1 - \hat{p})$
- **Entropy:** $H(\mathcal{S}) = -\hat{p} \log_2 \hat{p} - (1 - \hat{p}) \log_2 (1 - \hat{p})$

Both metrics satisfy $G(\mathcal{S}) = H(\mathcal{S}) = 0$ when $\hat{p} \in \{0, 1\}$ (a completely pure node) and reach their global maximum at $\hat{p} = \frac{1}{2}$ (maximal uncertainty). Crucially, both are strictly concave functions of $\hat{p}$.

Definition 3.2 (Candidate Split and Information Gain)

A candidate split on feature j at threshold τ partitions the node dataset $\mathcal{S}$ into two disjoint child subsets:

$$\mathcal{S}_L = \{i \in \mathcal{S} : x_{ij} \leq \tau\} \quad \text{and} \quad \mathcal{S}_R = \{i \in \mathcal{S} : x_{ij} > \tau\}.$$

Let $n = |\mathcal{S}|$, $n_L = |\mathcal{S}_L|$, and $n_R = |\mathcal{S}_R|$. The expected post-split impurity (also called the *conditional impurity*) $Q(j, \tau)$ is the sample-weighted average of the child node impurities:

$$Q(j, \tau) = \frac{n_L}{n} H(\mathcal{S}_L) + \frac{n_R}{n} H(\mathcal{S}_R).$$

The reduction in impurity achieved by performing this split is defined as the **information gain**:

$$\Delta(j, \tau) := H(\mathcal{S}) - Q(j, \tau).$$

(Note: The identical conditional expression $Q(j, \tau)$ and gain $\Delta(j, \tau)$ apply when evaluating impurity using Gini $G(\mathcal{S})$ instead of Entropy $H(\mathcal{S})$.)

Proposition 3.1 (Splitting never increases expected impurity)

For any candidate split on feature j at threshold τ , the information gain is non-negative:

$$\Delta(j, \tau) \geq 0,$$

holding universally for both entropy H and Gini impurity G .

³A set of training points $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^n$ refers to the collection of observations present at a specific node, where each point pairs a feature vector $x_i \in \mathbb{R}^p$ (e.g., passenger class, age, fare) with its observed survival status $y_i \in \{0, 1\}$. At the root node, $\mathcal{S}$ contains the full training set; at deeper nodes, $\mathcal{S}$ represents the conditional subset of passengers that satisfy all split criteria leading to that branch.

Proof.

Let $\hat{p}$, $\hat{p}_L$, and $\hat{p}_R$ be the empirical survival rates in $\mathcal{S}$, $\mathcal{S}_L$, and $\mathcal{S}_R$, respectively. By the law of total probability, the parent node's survival rate is the weighted average of its children's rates:

$$\hat{p} = \frac{n_L}{n} \hat{p}_L + \frac{n_R}{n} \hat{p}_R.$$

Because the entropy function $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ is strictly concave in $p \in (0, 1)$, Jensen's inequality⁴ applies directly:

$$H(\mathcal{S}) = H\left(\frac{n_L}{n} \hat{p}_L + \frac{n_R}{n} \hat{p}_R\right) \geq \frac{n_L}{n} H(\hat{p}_L) + \frac{n_R}{n} H(\hat{p}_R) = Q(j, \tau).$$

Subtracting $Q(j, \tau)$ from both sides yields $\Delta(j, \tau) = H(\mathcal{S}) - Q(j, \tau) \geq 0$. The identical derivation holds for Gini impurity $G(p) = 2p(1-p)$, which is also strictly concave. $\square$

Proposition 3.1 is what makes greedy tree construction well-posed: at every node there is a well-defined, non-negative quantity to search over, $(j^*, \tau^*) = \operatorname{argmax}_{j, \tau} \Delta(j, \tau)$, and the search terminates (rather than cycling) because impurity is monotonically driven toward zero as splits accumulate.

Definition 3.3 (Recursive Tree Construction and Leaf Prediction)

Starting at the root node initialized with the full training set $\mathcal{S}_{\text{root}} = \mathcal{S}$, a decision tree $T(x)$ is constructed recursively as follows:

1. **Evaluate Stopping Conditions:** At the current node with training subset $\mathcal{S}$, check if any terminal criteria hold:

- The node is pure ($H(\mathcal{S}) = 0$ or $G(\mathcal{S}) = 0$, meaning all labels y_i are identical);
- The current depth equals the maximum allowed depth ($\text{depth} \geq \text{max_depth}$);
- The number of samples is below the splitting threshold ($|\mathcal{S}| < \text{min_samples_split}$).

If any condition is satisfied, declare the node a **leaf node** ℓ and assign it a fixed class prediction $c_\ell \in \{0, 1\}$ based on the empirical majority vote within $\mathcal{S}$:

$$c_\ell = \mathbf{1}[\hat{p}(\mathcal{S}) \geq \tfrac{1}{2}] = \mathbf{1}\left[\frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} y_i \geq \tfrac{1}{2}\right].$$

2. **Greedy Split Search:** Otherwise, search across all features $j \in \{1, \dots, p\}$ and possible split thresholds $\tau \in \mathbb{R}$ to find the optimal split parameters (j^*, τ^*) that maximize the information gain:

$$(j^*, \tau^*) = \operatorname{argmax}_{j, \tau} \Delta(j, \tau).$$

3. **Partition and Recurse:** Partition $\mathcal{S}$ into child subsets $\mathcal{S}_L = \{i \in \mathcal{S} : x_{ij^*} \leq \tau^*\}$ and $\mathcal{S}_R = \{i \in \mathcal{S} : x_{ij^*} > \tau^*\}$, and repeat the process recursively on both $\mathcal{S}_L$ and $\mathcal{S}_R$.

When the decision tree finishes growing, it divides the feature space $\mathbb{R}^p$ into L separate sections, $R_1, \dots, R_L$, corresponding to its L leaves.

To predict whether an unseen passenger $x \in \mathbb{R}^p$ survives, the tree traces x down its decision rules to a single leaf region R_k and returns that region's majority vote $c_k \in \{0, 1\}$:

$$T(x) = \sum_{\ell=1}^L c_\ell \cdot \mathbf{1}[x \in R_\ell].$$

⁴Jensen's inequality states that for any concave function f and convex weights $\lambda_1, \lambda_2 \geq 0$ summing to 1, $f(\lambda_1 x_1 + \lambda_2 x_2) \geq \lambda_1 f(x_1) + \lambda_2 f(x_2)$.

Because the leaf regions $\{R_1, \dots, R_L\}$ form a strict partition of the feature space (they are mutually exclusive and cover all possibilities), passenger x belongs to exactly one region—say R_k . Consequently, the indicator function $\mathbf{1}[x \in R_\ell]$ equals 1 for $\ell = k$ and 0 for all $\ell \neq k$.

Expanding the summation demonstrates this cancellation explicitly:

$$T(x) = c_1 \cdot \mathbf{1}[x \in R_1] + \dots + c_k \cdot \mathbf{1}[x \in R_k] + \dots + c_L \cdot \mathbf{1}[x \in R_L]$$

which simplifies instantly to:

$$T(x) = c_1 \cdot 0 + \dots + c_k \cdot 1 + \dots + c_L \cdot 0 = c_k.$$

Remark 3.1 (Bias–Variance Tradeoff in Single Decision Trees)

A decision tree grown without stopping constraints (such that every leaf node is completely pure, $H(\mathcal{S}) = 0$) achieves zero training error and exhibits **very low bias**,⁵ as it can fit highly intricate relationships by creating fine, detailed rules.

However, such deep trees suffer from **high variance**.⁶

- A slight change in the training dataset can alter the greedy choice of feature j^* and threshold τ^* near the root node;
- Because lower branches inherit this initial decision, the changes compound recursively down the tree, resulting in a completely different set of prediction rules.

Hyperparameters like `max_depth` (limiting how deep the tree can grow) and `min_samples_split` (requiring a minimum number of samples to allow a split) act as structural regularizers. They trade a small increase in bias for a significant reduction in variance by preventing the tree from memorizing noisy patterns in the training data, ensuring the model generalizes well to unseen test passengers rather than performing perfectly on training data at the expense of test accuracy.

3.2 Random Forest: Averaging Decorrelated Trees

Single decision trees are prone to overfitting and high variance because small changes in the training data can completely change the tree’s predictions. A **Random Forest** fixes this by training many trees on different random subsets of the data and restricting each split to a random subset of features. Averaging predictions across these trees lowers variance and yields far more stable predictions on unseen data.

Definition 3.4 (Bootstrap Sample)

To construct tree $b \in \{1, \dots, N\}$, we create a resampled dataset $\mathcal{D}^{(b)}$ of size n from the original training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ by drawing n observations uniformly at random *with replacement*:

$$i_1^{(b)}, \dots, i_n^{(b)} \stackrel{\text{iid}}{\sim} \text{Uniform}\{1, \dots, n\} \quad \implies \quad \mathcal{D}^{(b)} = \left\{ \left(x_{i_k^{(b)}}, y_{i_k^{(b)}} \right) \right\}_{k=1}^n.$$

Because sampling is performed with replacement, $\mathcal{D}^{(b)}$ contains exact duplicate copies of some original observations while omitting others entirely.

⁵**Bias** measures the systemic error introduced by simplifying assumptions in a model. In this context, low bias means a deep tree makes very few restrictive assumptions about passenger survival, allowing it to learn complex, non-linear patterns (such as interactions between age, ticket class, and passenger title).

⁶**Variance** measures how sensitive a model’s predictions are to small fluctuations in the training dataset. In this context, high variance means a tree trained on a slightly different subset of Titanic passengers will pick different split thresholds near the top, creating entirely different rules down the branches and making erratic predictions on unseen test passengers.

Lemma 3.1 (The 63.2% Rule of Bootstrap Inclusion)

As $n \rightarrow \infty$, the probability that a specific training observation (x_i, y_i) is included in a given bootstrap sample $\mathcal{D}^{(b)}$ converges to $1 - e^{-1} \approx 0.632$.

Proof.

Consider a fixed observation i . In a single uniform draw from n items, the probability of *not* picking observation i is $1 - \frac{1}{n}$. Because the n draws are independent with replacement, the probability that observation i is omitted across all n draws is:

$$P(i \notin \mathcal{D}^{(b)}) = \left(1 - \frac{1}{n}\right)^n.$$

Taking the limit as $n \rightarrow \infty$, we obtain the fundamental limit definition of the exponential constant:

$$\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n = e^{-1} \approx 0.368.$$

The probability of inclusion is simply the complement: $1 - e^{-1} \approx 0.632$. □

Remark 3.2 (Out-of-Bag Implications)

Lemma 3.1 establishes that each tree in a Random Forest is trained on roughly 63.2% of the original data. The remaining $\approx 36.8\%$ of unseen observations provide an intrinsic validation set for computing unbiased error estimates without requiring a separate cross-validation split.

Definition 3.5 (Random Forest Ensemble)

A Random Forest consists of an ensemble of N decision trees $\{T_1, \dots, T_N\}$. Each tree T_b is constructed independently as follows:

1. Draw a bootstrap sample $\mathcal{D}^{(b)}$ of size n from the training set $\mathcal{D}$.
2. Grow a decision tree on $\mathcal{D}^{(b)}$ recursively. At each candidate split, randomly select a subset of $m_{\text{try}} = \lfloor \sqrt{p} \rfloor$ features⁷ out of all p features, searching for the optimal split (j^*, τ^*) exclusively within this subsampled feature set.

For a test sample $x \in \mathbb{R}^p$, each individual tree $T_b(x) \in \{0, 1\}$ outputs a binary prediction. The ensemble assigns its final prediction via majority vote:

$$\hat{y}(x) = \mathbf{1} \left[\frac{1}{N} \sum_{b=1}^N T_b(x) \geq \frac{1}{2} \right].$$

⁷ $\lfloor \sqrt{p} \rfloor$ denotes the floor function applied to $\sqrt{p}$, rounding down to the nearest integer. The square root choice is the standard in literature. For example, with $p = 10$ features, $m_{\text{try}} = \lfloor \sqrt{10} \rfloor = 3$ features are candidate-evaluated at each split.

4 Model Comparison: Decision Trees, Random Forests, and Logistic Regression

Choosing between a linear model like Logistic Regression and tree-based ensemble methods depends on the underlying structure of the data and the objective of the analysis.

1. **Decision Boundaries and Interactions:** Logistic Regression constructs a linear decision boundary in the log-odds space using the score $z = w^\top x + b$. It performs exceptionally well when the relationship between features and the target is approximately linear. However, it cannot capture complex, non-linear interactions (such as how passenger survival might depend on a specific combination of age, ticket class, and fare) unless interaction terms are manually engineered. In contrast, decision trees and Random Forests naturally partition the feature space into separate regions, allowing them to capture intricate non-linear patterns.
2. **Interpretability vs. Predictive Capacity:** A single decision tree is easy to interpret because its decision paths form direct, step-by-step logic rules. Logistic Regression is also interpretable because each weight w_j directly measures how feature x_j changes the predicted log-odds. A Random Forest sacrifices this direct interpretability for higher accuracy.
3. **Sensitivity to Scaling and Outliers:** Logistic Regression is sensitive to feature scaling and outliers, making data standardization crucial. Tree-based models are entirely invariant to monotonic transformations of features; scaling numerical columns like **Age** or **Fare** has no effect on impurity calculations. Trees are also robust to outliers because splits depend only on whether a feature value is above or below a threshold, regardless of how extreme that value is.

Remark 4.1 (Practical Selection Guidelines for Survival Prediction)

For the Titanic passenger survival task, Logistic Regression serves as a robust, low-variance baseline that reveals linear trends (e.g., overall survival advantages for women and first-class passengers). However, Random Forests generally achieve better predictive accuracy making them the preferred choice when predictive performance outweighs the requirement for a simple parametric formula.

5 Discussion: Two Families, One Question

Two structural claims recur across the preceding sections and are worth stating together rather than leaving implicit.

1. **Both families answer the same question with different mechanisms.** Every model in this document must decide how much to let itself fit the specific 891 passengers in the training set, as opposed to the population they were drawn from. Logistic regression answers this question inside its objective: Section 2 shows that L2, L1, and Elastic Net are three distinct penalty terms added directly to the cross-entropy loss (Corollary 2.2 and remarks 2.3 and 2.5), each shrinking or zeroing weights in a different way. Tree-based models answer the same question structurally rather than by penalizing an objective: a single decision tree is restrained by stopping rules such as `max_depth` and `min_samples_split` (Remark 3.1), while Random Forest restrains it by averaging many independently high-variance trees rather than by changing what any individual tree tries to do (Definitions 3.4 and 3.5). Lemma 3.1’s 63.2% bootstrap-inclusion figure quantifies exactly how much independence each resample buys, which is what makes that averaging effective.
2. **The two families do not carry the same optimization guarantee.** Theorem 2.3 shows that the cross-entropy loss — with or without an L2 penalty added, since a convex function plus a convex penalty remains convex — is convex on the full parameter space, so gradient descent in Section 2 is guaranteed to reach a global optimum regardless of where it starts. No comparable guarantee exists for the greedy split search in Definition 3.3: Proposition 3.1 only guarantees that each individual split does not increase expected impurity, not that the resulting sequence of locally optimal splits is globally optimal across the whole tree. This is the sense in which logistic regression and tree-based models are genuinely different kinds of optimization problem, and it is also why Section 4’s practical tradeoffs — a stable, interpretable linear boundary against a flexible, harder-to-certify partition of the feature space — are not an accident of implementation but a direct consequence of the theory developed in Sections 2 and 3.

6 Empirical Validation on Real Data

To evaluate the empirical behavior of parametric linear models against non-parametric tree architectures, all models were trained and evaluated on the Kaggle Titanic dataset ($n_{\text{train}} = 891$, $n_{\text{test}} = 418$). Model hyperparameters were tuned on an 80/20 stratified train-validation split before retraining on the full training set.

Model	Train Acc.	Test Survivors ($n = 418$)	Kaggle Score
Logistic Regression (Unregularized MLE)	83.05%	161	0.77751
Logistic Regression (Ridge L_2)	83.39%	159	0.77751
Logistic Regression (Lasso L_1)	83.39%	159	0.77751
Logistic Regression (Elastic Net)	83.39%	159	0.77751
Decision Tree (Single)	85.30%	152	0.77511
Random Forest (Ensemble)	83.50%	163	0.77751

Key Empirical Observations

1. **High Inter-Model Consensus:** Regularized linear variants and the Random Forest ensemble converge to an identical Kaggle test score of **0.77751**. The Random Forest matches

Logistic Regression predictions on 98.56% of unseen test passengers, confirming that feature engineering captured the primary underlying survival boundaries across both linear and non-linear model families.

2. **Variance Reduction via Ensembling:** The unconstrained single Decision Tree overfits slightly to training noise (85.30% train accuracy), dropping its test score to **0.77511**. Constraining tree depth (`max_depth = 3`) and averaging predictions in the Random Forest reduces model variance, bringing training accuracy to a realistic 83.50% and recovering top-tier test performance (0.77751).
3. **Primary Survival Drivers Across Architectures:** Interpreting model parameters across both families reveals three consistent structural drivers of survival probability:
 - *Demographic Evacuation Priority:* Explicit indicators like `WomanOrChild` and `Sex` provided the largest positive linear weights (+0.574 and +0.519) and dominated initial decision tree partitions, directly reflecting historical priority protocols.
 - *Socioeconomic Advantage:* `Pclass` and `Fare` consistently served as primary secondary splitters in the Random Forest (10.71% importance for `Fare`) and strong linear penalties (−0.545 for lower classes), capturing the survival advantage of higher deck access.
 - *Negative Penalties:* Holding an adult male title (`Title_Mr`) or traveling in a large group (`LargeFamily`) acted as the strongest negative predictors across all algorithms, penalizing survival odds due to lower evacuation priority and group coordination difficulties.